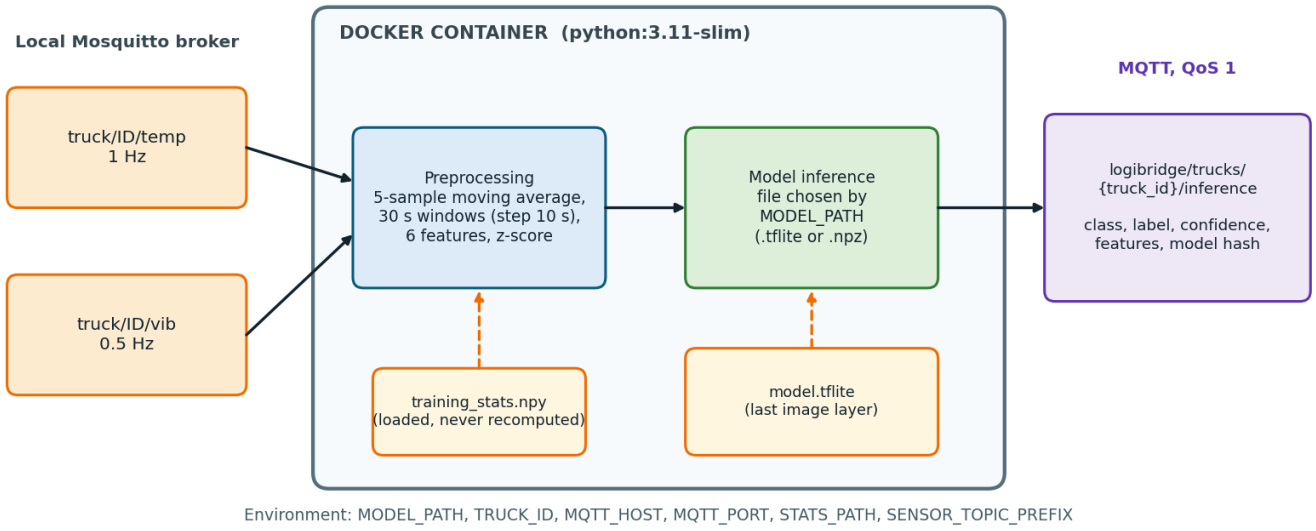
Task D2 — Docker Containerisation and OTA Demo

Inference container and layer-cached model updates | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component D, Module 4

At a glance

Item	Result
Container	`python:3.11-slim`; preprocessing → model inference → MQTT publish to `logibridge/trucks/{truck_id}/inference`
Layer order	Every pip install layer comes before `COPY model.tflite .`, which is the last layer that carries data
Model switching	`MODEL_PATH` selects the model at run time; no rebuild
OTA layer cache	A model-only change rebuilds exactly one data layer (predicted by applying Docker's cache rule to the real Dockerfile)
Bandwidth, 85 trucks	About 5,523.6 MB saved of 5,525 MB (99.98%), Rs 552.36 at Rs 0.10/MB. Image sizes are my estimates; replace with measured values.
Status	**Not built or run on Docker, and no model.tflite yet:** Docker and TensorFlow were not available to me. All 174 project tests pass for the logic (Section 7).

1 What the container does



Variable	Default	Purpose
MODEL_PATH	/app/model.tflite	Model file (.tflite, or .npz weights). Change it to switch variants without rebuilding
TRUCK_ID	T001	Used in the output topic logibridge/trucks/{truck_id}/inference
MQTT_HOST, MQTT_PORT	localhost, 1883	Local Mosquitto broker (in compose: mosquitto)
STATS_PATH	/app/training_stats.npy	Saved normalisation statistics (loaded, never recomputed)
SENSOR_TOPIC_PREFIX	truck/{truck_id}	Where the sensors publish; the service reads <prefix>/temp and <prefix>/vib as in Task C1
REFERENCE_DIST_PATH, DRIFT_SCORE	unset, normal_confidence	PSI drift monitoring (Task E1); off unless a reference file is given

```
logibridge/trucks/T001/inference  (QoS 1)   [example message]
{"truck_id":"T001","window_end":"2027-01-15T08:00:30.000Z","class":2,"label":"Critical","confidence":0.9998,
 "probabilities":[0.0,0.0002,0.9998],"features":{"temp_mean":5.1,...,"vib_kurtosis":0.8},
 "model":{"file":"model.tflite","sha256":"3f9c1a2b7d40"}}
```

2 Dockerfile and layer order

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    MODEL_PATH=/app/model.tflite \
    STATS_PATH=/app/training_stats.npy \
    TRUCK_ID=T001 \
    MQTT_HOST=localhost \
    MQTT_PORT=1883
RUN useradd --system --uid 10001 --no-create-home logiedge
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY preprocessing.py drift_monitor.py inference_service.py training_stats.npy ./
COPY model.tflite .
USER logiedge
CMD ["python", "inference_service.py"]
```

Step	Holds	Changes when	Model-only change
FROM, ENV, useradd, WORKDIR	Base image, settings, non-root user	Rarely	Cached
COPY requirements.txt	Dependency list	A package is added or updated	Cached
RUN pip install	numpy, paho-mqtt, tflite-runtime	requirements.txt changes	Cached
COPY code + statistics	preprocessing.py, inference_service.py, training_stats.npy	Code or statistics change	Cached
COPY model.tflite .	The model	Every model update	**Rebuilt: the only data layer**
USER, CMD	Metadata	Never	Re-run, 0 bytes

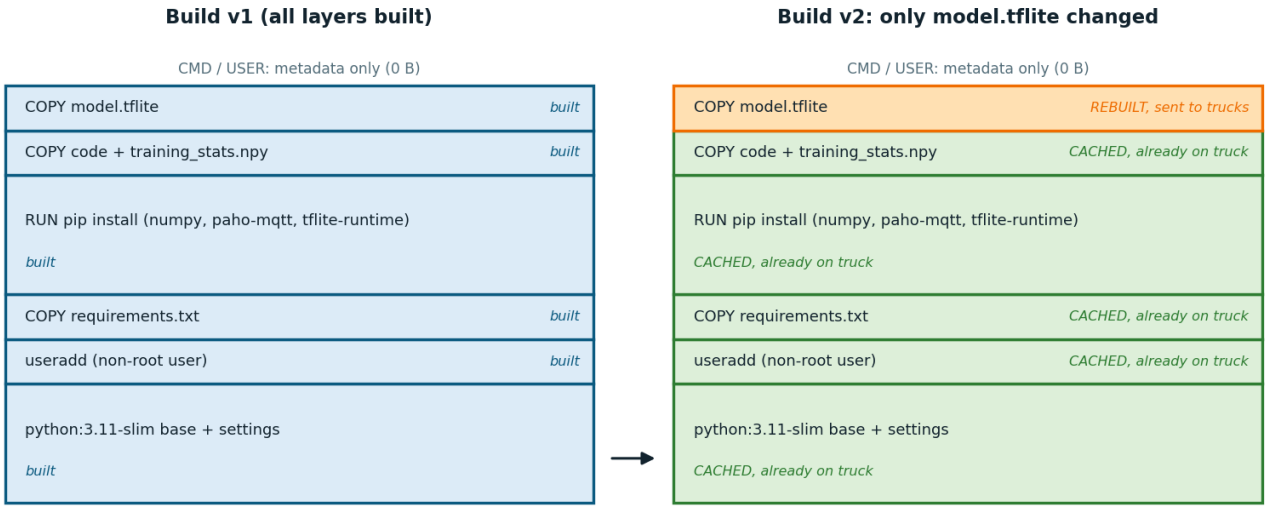
3 Switching model variants without a rebuild

```
# default model baked into the image
docker run --rm --network host logiedge/inference:v1

# another variant from a mounted folder, same image, no rebuild
docker run --rm --network host -v "$PWD/models:/models:ro" \
           -e MODEL_PATH=/models/model_onset.tflite logiedge/inference:v1
```

The model hash in each result message (`model.sha256`) shows which variant produced it, which makes the switch visible on screen. A wrong path stops the container with a clear error (exit code 2) instead of running with no model.

4 OTA layer-cache demonstration



Run it (needs Docker): `./ota_demo.sh` builds v1 with no cache, replaces only model.tflite, rebuilds v2, compares the image layer digests and prints the bandwidth figures. In the real build log, steps 1–7 show `CACHED` and only `COPY model.tflite .` is executed. The digest comparison should report all layers identical except one.

Prediction from the Dockerfile (Docker's cache rule applied by `layer_cache_check.py` to two build contexts that differ only in model.tflite):

Step	Instruction	Result
1–4	FROM, ENV, RUN useradd, WORKDIR	CACHED
5	COPY requirements.txt .	CACHED
6	RUN pip install -r requirements.txt	CACHED
7	COPY preprocessing.py inference_service.py training_stats.npy ./	CACHED
8	COPY model.tflite .	REBUILT (only data layer)
9–10	USER logiedge, CMD [...]	REBUILT (metadata, 0 bytes)

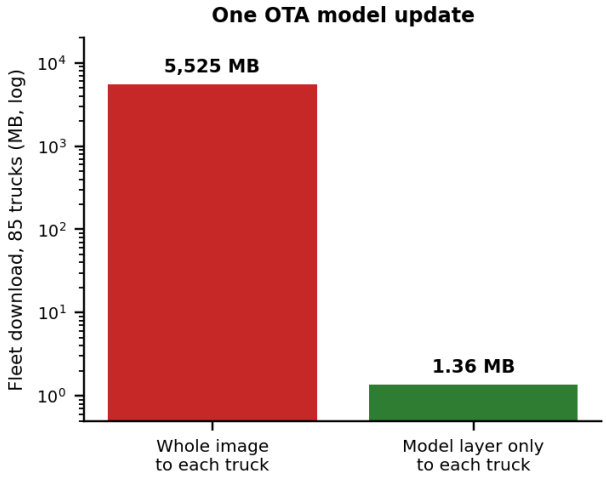
Condition for the saving: *trucks only skip the cached layers if the rebuilt image reuses the same layers, so build on the same builder or use a shared registry cache (`--cache-from`). A clean build on a new machine produces new layer digests and trucks would download everything.*

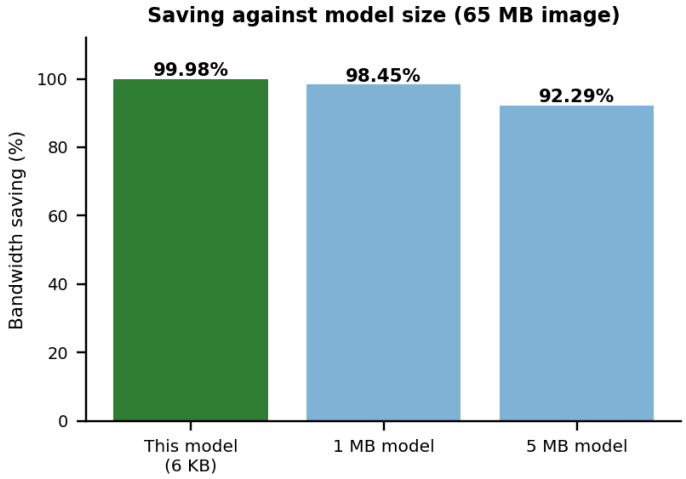
5 Bandwidth saving for 85 trucks

A truck that already runs the previous image downloads only the layers it lacks. With the model as the last layer, that is the new model layer plus a small manifest and config. Sizes below are my estimates, not measurements: `ota_demo.sh` measures the real ones and `ota_bandwidth.py` recomputes everything.

Assumption	Value	Basis
Full image, compressed	65 MB (range 62–67)	python:3.11-slim about 45–46 MB (copies on Docker Hub: 45.3 MB arm64, 45.7 MB amd64) + numpy wheel 14.2 MB aarch64 or 18.3 MB x86_64 (PyPI) + tflite-runtime about 2–3 MB (size not found; assumed)
Model layer, compressed	6 KB	803 float32 parameters (3.2 KB) plus file structure (estimate)
Manifest and config per update	10 KB	Estimate
Tariff, units	Rs 0.10 per MB; 1 MB = 1,000,000 bytes	As in Task A1

Step	Calculation	Result
Full update per truck	Whole image	65.000 MB
Full update, 85 trucks	85 × 65 MB	5,525.00 MB; Rs 552.50
Layer update per truck	6 KB + 10 KB = 16 KB	0.016 MB
Layer update, 85 trucks	85 × 0.016 MB	1.36 MB; Rs 0.14
Saving, 85 trucks	5,525.00 − 1.36	**5,523.64 MB (99.98%); Rs 552.36**





The saving shrinks as the model grows (a 1 MB model saves 98.45%, a 5 MB model 92.29%) but stays large. Across the 62–67 MB range the saving is 5,269 to 5,694 MB (99.97% to 99.98%). For scale, one full 65 MB image pull equals about 1,916 days of a truck's alert traffic from Task A1 (0.0339 MB per day). At 265 trucks the same update saves 17,221 MB.

6 Demo video script (about 3 minutes)

I cannot record video, so this is a ready-to-film script. Replace the numbers with those printed by `ota_demo.sh` if they differ.

Time	On screen	Say
0:00	Dockerfile	The image starts from python:3.11-slim. All pip installs come first, the model is copied last, and MODEL_PATH lets us pick a model at run time.
0:25	docker compose up; mosquitto_sub on logibridge/trucks/#; simulator running	The container preprocesses the sensor stream, runs the model and publishes each result to logibridge/trucks/T001/inference.
1:00	Run with -e MODEL_PATH=/models/model_onset.tflite	Same image, different model, no rebuild: the model hash in the messages changes.
1:25	./ota_demo.sh: build v1, replace model.tflite, build v2	Now an over-the-air update. I change only the model file and rebuild.
1:55	Build log with CACHED lines; layer-digest comparison	Every step is cached except COPY model.tflite. All layers are identical except one, so a truck downloads only that layer.
2:25	ota_bandwidth.py output	For 85 trucks, sending the whole image would be about 5,525 MB. Sending only the model layer is about 1.36 MB. That saves 5,524 MB, 99.98%, about Rs 552 at Rs 0.10 per MB.

7 Verification: what ran and what did not

Check	How	Result
Dockerfile rules: base image, pip before model, model last data layer, MODEL_PATH, non-root user, copy sources, .dockerignore, pinned requirements	Static tests	Pass
Docker cache rule on the real Dockerfile: model-only change rebuilds one data layer; code change also rebuilds the model; requirements change rebuilds pip	Cache analysis on two build contexts	Pass
Service: topic, payload, QoS 1, subscriptions, last will and status, malformed input, failing model, clean stop	paho stub	Pass
MODEL_PATH switching: two variants give different model hashes; startup errors return code 2	Tests	Pass
numpy backend gives the same classes as the training code; results follow the simulator mode	Tests	Pass
TFLite path: float32 input and output; int8 input quantised and output dequantised	Fake interpreter	Pass
Bandwidth arithmetic	Tests	Pass
Package pins exist with Python 3.11 wheels for x86_64 and aarch64: numpy 1.26.4, paho-mqtt 2.1.0 (pure Python), tflite-runtime 2.14.0; tflite-runtime needs numpy 1.23.2 or newer and glibc 2.34+ on aarch64, which python:3.11-slim (Debian 13) has	Checked against PyPI and Docker Hub listings; not installed	Checked
TensorFlow 2.14 supports Python 3.9–3.11 and needs numpy 1.23.5 or newer; the "FULLY_CONNECTED version 12" mismatch error between a new converter and an old runtime is a documented failure	Checked in project files and issue reports	Checked
paho 2.x callback signatures (on_connect, on_disconnect) match the code; eclipse-mosquitto:2 needs a mounted config with a listener and allow_anonymous for the demo; BuildKit plain output prints "CACHED"	Checked in documentation	Checked
ai-edge-litert fallback: cp311 wheels exist for x86_64 and aarch64 (glibc 2.17+)	Checked against PyPI	Checked
docker build, real layer cache, image size	Needs Docker	Not run
convert_to_tflite.py and the real TFLite runtime	Needs TensorFlow	Not run
Live Mosquitto and an actual pip install	Needs network and broker	Not run

8 Items to confirm

- **Create model.tflite first:** in a Python 3.11 environment run `pip install tensorflow==2.14.0 numpy==1.26.4 tflite-runtime==2.14.0` then `python3 convert_to_tflite.py`. It refuses to write the file unless the TFLite model matches the numpy model on every dataset window. Tell me the result and I will fix anything that fails.
- **Topic names.** The output topic follows your specification (logibridge/trucks/...), while the sensors still publish on truck/ID/... from Task C1. Set SENSOR_TOPIC_PREFIX if you move them. Please confirm the "logibridge" spelling is intended.
- **Package pins (checked online).** numpy 1.26.4 has Python 3.11 wheels for x86_64 and aarch64. paho-mqtt 2.1.0 is pure Python and the latest release. tflite-runtime 2.14.0 has Python 3.11 wheels for x86_64, aarch64 and armv7l and needs numpy 1.23.2 or newer. numpy stays below 2.0 because that tflite-runtime (October 2023) predates NumPy 2.0; this last point is my inference, not a reported failure. TensorFlow 2.14 needs Python 3.9–3.11 and numpy 1.23.5 or newer with no upper bound, so keep the numpy pin when installing it. I did not run pip install.
- **Two risks from that check.** (1) tflite-runtime 2.14.0 is the last release, so convert with TensorFlow 2.14 (it has Python 3.11 wheels but none for 3.12), because a model from a much newer TensorFlow can fail to load on the older runtime; convert_to_tflite.py now also tests with tflite-runtime if installed. (2) The aarch64 wheel needs glibc 2.34 or newer, which python:3.11-slim provides (it is currently Debian 13); do not switch to an Alpine or Debian 11 base. If tflite-runtime gives you trouble, ai-edge-litert has Python 3.11 wheels for x86_64 and aarch64, at the cost of a larger image.
- **Sizes.** The base-image and numpy sizes now come from published listings (about 62–67 MB in total). The tflite-runtime wheel, the 6 KB model layer and the 10 KB manifest are still estimates, so run ota_demo.sh to replace them with measured values before filming.
- **INT8 later.** Normalised inputs run from about −3 to 1,300, so full-integer quantisation can crush the Normal range; validate accuracy before shipping a quantised model.